

AgentWorkShop: A Governed Supervisory Execution Layer for Industrial Agent Teams

Anonymous Authors

Abstract—Supervisory optimization above the regulatory control layer—tuning recipes, schedules, and setpoints against quality, throughput, energy, or discharge targets—is repeated in every plant; so is the integration work around it, because each deployment re-establishes which actor may change which parameter, which limits apply to the current product and run, whether the device accepted the command, and whether the process improved. This paper presents AgentWorkShop, a governed supervisory execution layer that turns those repeated concerns into one path for agent teams: each intervention is one governed transaction, with admission, approval, actuation, readback, observation, verdict, and compensation recorded as separate evidence states instead of one success flag. Team identities connect to semantic data-acquisition (DAQ) and write-control (DCW) nodes through explicit capabilities; line, product, recipe, and run context is resolved at execution time; and six protocol families, self-describing node and driver contracts, versioned production objects, plugin extension points, and a typed software development kit (SDK) form the integration surface, while process models, objectives, guards, and control policies remain external inputs. The evaluation exercises this single governed path through protocol closure and governance attribution, scripted-policy agent-team missions, a deterministic closed loop against an offline optimum, and concurrent process missions: the governed path intercepts every configured out-of-window probe without blocking an admissible write, both missions close their governed loops, and the process missions reach their archived predicates, while task completion, readback, and attainment stay separately recorded. The results support an inspectable, simulation-tier architecture for industrial supervisory execution.

Index Terms—Agent teams, industrial informatics, recipe optimization, write governance.

I. INTRODUCTION

INDUSTRIAL optimization occurs above the regulatory control layer: engineers adjust recipes, schedules, and setpoints to meet quality, throughput, energy, or discharge objectives while PLCs and local controllers retain fast regulation. The supervisory decision, however, is only one part of the intervention: a usable execution path must also establish who may change which parameter, which limits are active, whether the device accepted the command, and whether the process improved. Because these concerns are normally embedded in application code, they are rebuilt for every new process, protocol, or agent framework, and the rebuild is where duplicated drivers, ad-hoc permission checks, and logs that cannot be tied back to a decision accumulate.

These concerns recur across plants whose physics and objectives differ— injection molding coupling part weight to pressure, time, and temperature; wastewater treatment (WWTP) balancing effluent constraints against aeration and dosing cost; annealing trading material properties against line

speed; and BOPET (biaxially oriented polyethylene terephthalate) coordinating casting and stretching sections. All need an auditable bridge from a team-level task to heterogeneous acquisition and control nodes.

Large language model (LLM) agents are being studied for industrial diagnosis, engineering assistance, and PLC-program generation [1–4], including simulated controller and machine interaction [5, 6]. Yet orchestration frameworks and protocol connectivity do not by themselves answer who is authorized, which production limits apply, or how a write is linked to later evidence (Section II). Moreover, a task state such as COMPLETED can coexist with a missed process objective.

AGENTWORKSHOP addresses this execution gap through member–node bindings and shared line, product, recipe, and run context. The response is organizational rather than algorithmic: each intervention is treated as one governed transaction—a member proposes a value, admission intersects the hard, baseline, product, and recipe limits, approval, dispatch, readback, observation, and the caller’s verdict follow, and compensation can reverse the change—each step left as its own evidence record, so a completed task never stands in for a verified process outcome. Nodes expose engineering quantities rather than register addresses; bindings scope sanctioned tools. Fig. 1 contrasts this organization with application-specific integration; fast regulation, certified protection, and the choice of optimization algorithm remain outside the framework.

The paper makes four contributions:

- 1) a **capability-based industrial execution architecture** connecting agent-team identity, semantic data-acquisition (DAQ) and write-control (DCW) nodes, heterogeneous protocol drivers, and line–product–recipe–run context (run context gates dispatch; evidence records are time-windowed, Section III), with task membership distinct from physical-write authority;
- 2) a **reusable integration boundary** of self-describing node and driver contracts across six protocol families (five exercised in the evaluation), versioned production objects, plugin extension points, and a typed software development kit (SDK), into which the evaluated reference adapter projects scenario blueprints while process models and policies remain external;
- 3) an **evidence-preserving governed intervention workflow** separating admission, optional approval, actuation, readback, observation, the caller’s verdict, compensation, and backstop actions, so a completed task never stands in for a verified process outcome; and
- 4) a **layered evaluation** exercising one governed path through protocol closure with mechanism ablation,

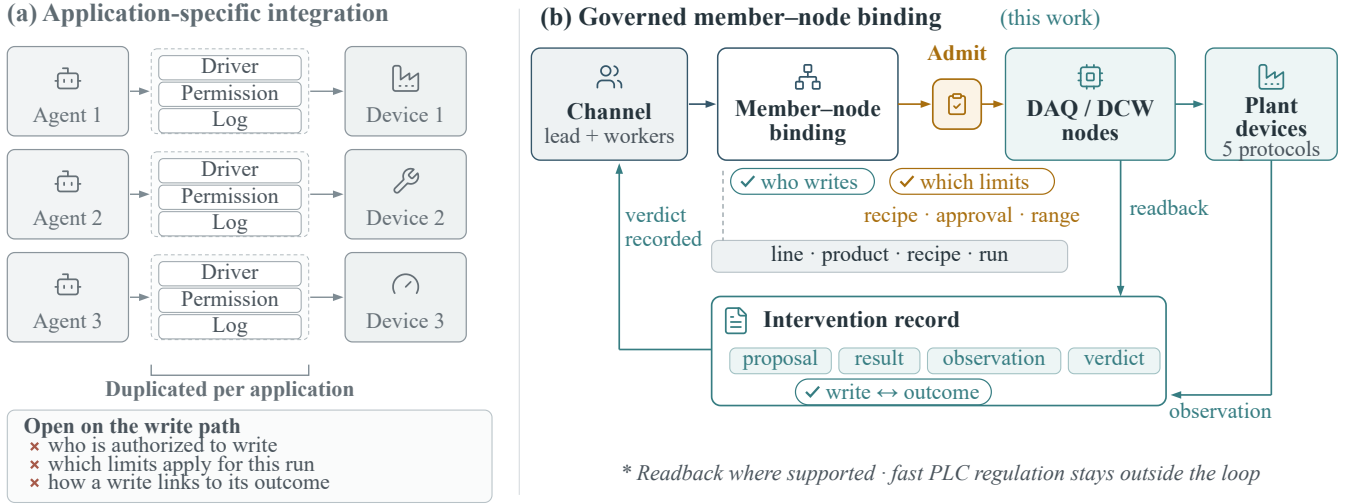


Fig. 1. Conceptual supervisory integration: (a) application-specific integration duplicating drivers, permissions, and logging per application; (b) governed member-node binding in AGENTWORKSHOP, where requests pass recipe, approval, and range checks and intervention records return outcome, observation, and verdict. Patterns are compared, not optimization performance; PLC regulation remains outside the agent-team loop.

agent-team missions, a deterministic controller loop, and concurrent process missions, with task state and objective attainment reported separately throughout.

Scope is stated once here and revisited in Section VI: real software protocol stacks execute against author-implemented dynamic simulators, scripted policies drive the governed tools for repeatability with two live engine turns reported separately, and fast regulation, certified protection, physical-plant transfer, and optimizer ranking are outside the claims made here. The remainder of the paper covers related work, the architecture, the governed intervention mechanisms, the evaluation, and discussion with limitations (Sections II–VI).

II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Agent-based manufacturing has long connected organizational roles to industrial capabilities [7]. Recent LLM-oriented work spans industrial knowledge graphs [8], diagnosis [9], PLC generation and verification [3, 4], digital-twin configuration [10], OPC UA machine control [6], and simulated fault handling [5, 11], while InstructMPC combines language-derived objectives with model-predictive control [12]. These studies motivate agent-facing industrial tools; the shared execution layer carrying authority and evidence across scenarios—our focus—is complementary, not a new controller or reasoning method.

B. Orchestration, Industrial Models, and Governance

AutoGen, LangGraph, and CrewAI coordinate tasks and tools [13–15], and the Model Context Protocol (MCP) and Agent-to-Agent (A2A) specifications standardize tool and inter-agent interfaces [16, 17]. These frameworks coordinate reasoning and tool invocation, but the tool they expose is usually a bare function signature carrying no line, product, or recipe context, with no record separating admission from execution. ISA-88, ISA-95, ISO 23247, and IEC 63278 supply

complementary models for recipes, operations, and digital twins [18–21], and AGENTWORKSHOP’s versioned recipes and run dispatch instantiate a reduced subset of that stack: master-recipe-style versions over a line-product-run resource model, without batch procedure models or a full equipment hierarchy. The governed life cycle itself—bounded proposals, post-write holds, attributable intervention journals—transposes the disciplines of run-to-run control [22] onto a supervisory write path; the increment here is interchangeable strategies over heterogeneous protocols with evidence that survives strategy replacement. Industrial Internet of Things (IIoT) and shop-floor agent architectures likewise emphasize contextualized integration [23], and the nearest neighbors [24, 25] target control decoupling and task decomposition. A recent line instead enforces runtime guarantees on LLM agents—judge and guard agents, capability-based information flow, rule-based enforcement [26–28]; these gate software tool calls; none owns a physical write path with device readback and a write journal; and their policies could compile onto the enforcement substrate contributed here, whose differentiator is production-scoped authority and evidence.

Permissions, range checks, approvals, and audit trails have DCS/SCADA/MES and IEC 62443 counterparts [29], while formal shielding, control-barrier functions, and simplex run-time assurance offer stronger specified properties [30–32]. AGENTWORKSHOP instead implements source-dependent checks and a rule-based backstop and makes no formal protection guarantee: the backstop (Section IV-C) is a rule-based recovery action in the spirit of ISA-18.2 and EEMUA-191 [33, 34] rather than a run-time assurance bound. Human oversight likewise requires more than an approval dialog [35, 36]. Agent benchmarks [37–39], process-monitoring datasets [40, 41], and prompt-injection studies [42, 43] inform adjacent evaluations; ours separates source checks, protocol execution, mechanism ablation, controller trials, and multi-scenario missions. In sum, the coordination frameworks supply no production context, the standards no execution path, and

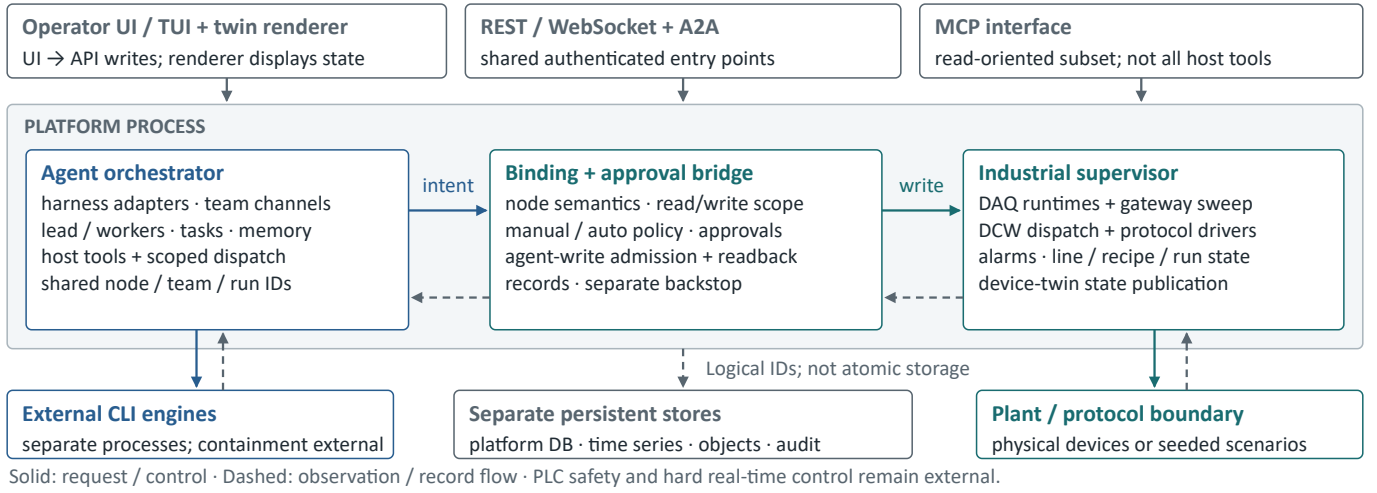


Fig. 2. System architecture and execution boundaries. The platform hosts team routing, binding and approval, industrial supervision, and records; operator interfaces sit above it, and agent engines, data stores, protocol endpoints, and the plant remain separate. Shared identifiers provide attribution but do not make storage and actuation transactional.

guard agents gate software tools; the open slot is the governed physical write path itself.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP links team coordination to plant interaction through industrial nodes and production context (Fig. 2). Channels coordinate deployed members and tasks; controller services mediate node operations; production objects identify the line, product, recipe, and run. Five planes separate the concerns: a *team* plane decides what to investigate and propose, an *integration* plane presents devices as semantic nodes, a *governance* plane decides whether a request is admissible, an *operations* plane supplies production context and evidence, and an *extensibility* plane admits drivers and services through sanctioned registration points. The decomposition is what makes the framework reusable: scenario knowledge sits in blueprints, signal maps, predicates, and driver configuration, not in team scheduler, approval service, or persistence code.

Each plane draws its own boundary between platform guarantees and scenario inputs: team coordination owns member identity, task state, and a member-written episodic memory retrieved by hybrid search; industrial integration owns protocol acceptance, readback, freshness, and transport errors, its node stream feeding both the agent and a live digital-twin view; governance owns admission, approval, actuation, write readback, observation, verdict, compensation, and rollback; production operations attaches line, product, recipe, and run context while PLC protection stays independent; and extensibility admits plugins, drivers, agent tools, panels, and SDK clients through sanctioned services. In every plane the common execution path is unchanged: what a scenario supplies is data and strategy. Fig. 2 draws these planes as services: the agent orchestrator is the team plane, the binding-and-approval bridge the governance plane, and the industrial supervisor the integration plane, and the operations and extensibility planes the persistent stores and sanctioned registration services around them.

A. Supervisory Core and Nodes

The Node.js core hosts channel routing, industrial services, and protocol adapters. A channel contains a lead, workers, tasks, messages, and retrievable memory, deployed through WebSocket, MCP, A2A, REST, CLI, or TUI entry points; engine adapters preserve member and node identity across runtimes, and channel membership alone confers no write authority.

DAQ nodes convert device readings to engineering quantities; DCW nodes translate engineering setpoints into driver commands and, where supported, readback. Per-node runtimes manage timing; gateway services supply connection state, recipe context, and write bookkeeping. The built-in semantic surface spans six protocol families—Modbus TCP, Modbus RTU-over-TCP, OPC UA, MQTT, HTTP/REST (client and server), and acquisition-only S7; the evaluation instantiates five of them as lines (Section V-C). The deployment reaches author-implemented dynamic simulators through the same node and driver contracts a plant would use; they are deterministic per seed, not validated digital twins. The first reuse boundary is declarative: a scenario declares signals, units, scaling, precision, and driver configuration, and the team loop is unchanged; fast regulation and independent protection stay in plant controllers.

Driver contracts are deliberately narrow: a driver declares how a semantic quantity maps to its transport payload, whether the endpoint is writable, and whether a readback is available, while the controller resolves member authority and active limits. Connection, transport, and protocol errors attach to the request rather than one success flag, and register addresses and byte order appear only here, so a new fieldbus family leaves the loop untouched.

Acquisition is supervisory rather than hard real-time: each DAQ node runs its own cadence under a rotating sweep, so a slow endpoint delays only its own samples. The verdict stage queries the timestamped, quality-stamped history rather than issuing a blocking read, which is why a windowed average can lag the device by one cadence.

B. Bindings, Context, and Reuse Boundaries

A member–node binding grants access to a sanctioned tool on a specific node and selects automatic or approval-mediated execution. Line permissions (none, read-only, operate) authorize binding creation and mutation; runtime invocation is authorized by the persisted capability, not channel membership. Intervals are re-checked every request instead of cached, so revoking a binding or switching the recipe takes effect on the next request. The same path serves a console, a harness, and an agent engine, because authority does not depend on the client; only the evidence record differs.

Production objects are versioned rather than overwritten: a recipe is dispatched against a known-good version, reverted non-destructively, and compared with the values actually applied, and an intervention is read against the definition that was active when it was admitted. Plugins register hooks, routes, agent tools, drivers, and panels only through sanctioned services, so an added driver inherits admission, readback, and evidence capture. A BOPET worker proposes casting-speed, fast-roll, or rail-width changes through bound DCW nodes, and the same mission pattern serves the other three scenarios with only blueprint, guards, and policy differing (Section V-F).

The second reuse boundary is a reference scenario-onboarding adapter (Section III-C); the SDK exposes the same surface to ordinary programs, and runtime settings are hot-reloadable. The stores share identifiers rather than a transaction domain. A device may accept a write even if bookkeeping fails, and records lack a run identifier, so time-range queries can cross runs unless bounded.

C. Reference Scenario-Onboarding Contract

The reference integration follows a bounded onboarding contract: the engineer declares a blueprint of devices, signals, units, roles, setpoints, and plant-model parameters; a differential “ensure” pass creates missing devices, repairs drift, verifies connectivity, loads the model without disturbing other lines, projects the blueprint into semantic nodes and a recipe, and tags the line for idempotent reuse; a channel is built from the common team template with the required bindings; and a mission supplies objective, guards, response windows, and policy. The adapter demonstrates reuse of platform services, not configuration-only commissioning of physical plants.

D. Governed Optimization Task Life Cycle

A governed optimization task is a life cycle rather than a single tool call, and AGENTWORKSHOP exposes each transition separately. The task enters a channel carrying goal, line, product, recipe, run, and task identifiers; a lead member claims it and hands it to a worker holding member–node bindings on the relevant nodes. The worker then repeats one fixed job: read the controlled quantity over a bounded window, propose a setpoint inside the resolved interval, invoke the governed write, wait for the response, and record a verdict (Section V-B instruments it).

Each iteration therefore leaves more than a written value: the write opens an optimization record carrying prior and requested values, the admitting interval, the acting member, and

timestamps; readback, where supported, closes the command path; the parameter journal attributes the change to its source; and attainment is computed against the scenario predicate while the task closes with a workflow terminal state; the per-intervention view of the same life cycle follows in Section IV. This bounds what “automatic optimization” means here: a controller, a scripted policy, a conventional optimizer, or an agent engine may occupy the strategy slot, while authority, evidence, and closure stay on the platform.

IV. GOVERNED EXECUTION MECHANISMS

A supervisory intervention must preserve distinctions that are often collapsed in an agent tool call—permission to propose, admission of a value, driver acceptance, readback, process response, a caller’s verdict, and compensation—and AGENTWORKSHOP exposes these as separate, distinguishable evidence states. Fig. 3 shows the runtime that carries these steps. One governed intervention proceeds: a task with goal, line, product, recipe, run, and task identifiers enters a channel and a lead dispatches it to a bound worker; authority resolves from the member–node binding, never from channel membership or prompt content; the proposal must lie in the Eq. (1) interval; on a manual binding a timed approval wait is inserted (fail-closed on expiry); the driver dispatches, the device accepts, and readback, where supported, closes the command path; the write opens an optimization record; the observation is drawn from the timestamped history and judged; compensation is a separately admitted write referencing the record it reverses; and the backstop evaluates records no verdict has closed. Every agent-originated path to the plant passes through the same governed write; the subsections below follow that path.

A. Task, Observation, and Admission

The strategy receives a task and goal in production context and decides what to propose and when to stop; the platform governs execution over semantic node parameters—pressure, dosing rate, line speed, rail width—in engineering units.

For an ordinary agent setpoint request on node n , the admission interval is

$$\mathcal{I}_n(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \quad (1)$$

where $\mathcal{H}_n$ is the hard engineering range and the remaining terms the baseline, active-product, and active-recipe limits; missing optional limits add no restriction. The interval bounds one parameter at a time; joint, cross-variable windows live in the scenario guards. The interval is not a learned feasibility region or a physical safety certificate; availability, binding state, and approval policy are checked separately, and it bounds magnitude, not intent: in-window but adverse sequences surface at judgment and compensation (Section V-F reports two).

B. Semantic Write and Source-Specific Routes

A semantic card exposes quantity, unit, precision, range, value, freshness, and recipe context. The controller resolves the binding and the limits; the driver then serializes the command.

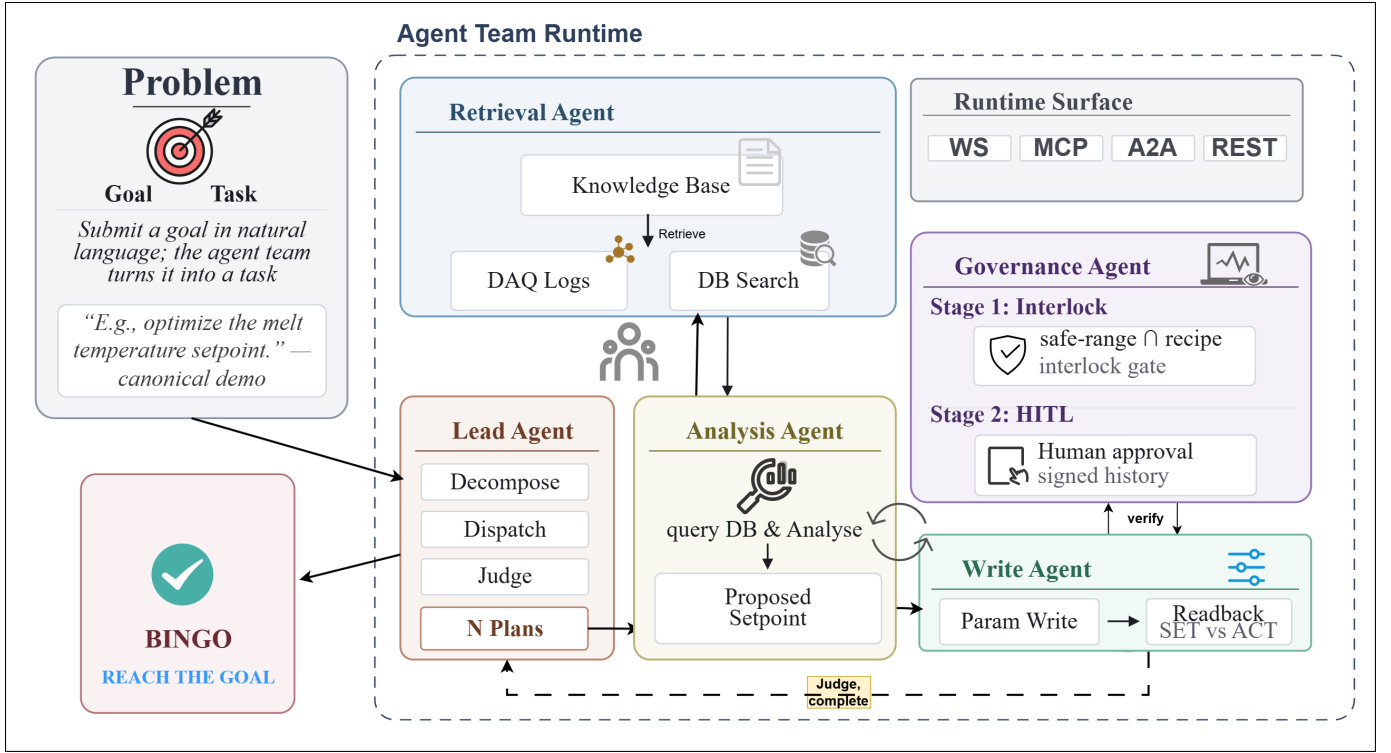


Fig. 3. Implemented agent-team runtime. A natural-language goal enters as a task; the lead agent dispatches one of N plans; the retrieval and analysis agents produce the proposed setpoint; the write agent performs the write and reads back the value actually present (SET versus ACT). The governance agent is a deterministic two-stage gate—Stage 1: the Eq. (1) admission (drawn simplified), Stage 2: the approval wait on manual bindings—both stages journaled with actor attribution. The judge/complete verdict returns to the lead agent; a reached goal ends the governed path, not evidence that the process objective was met.

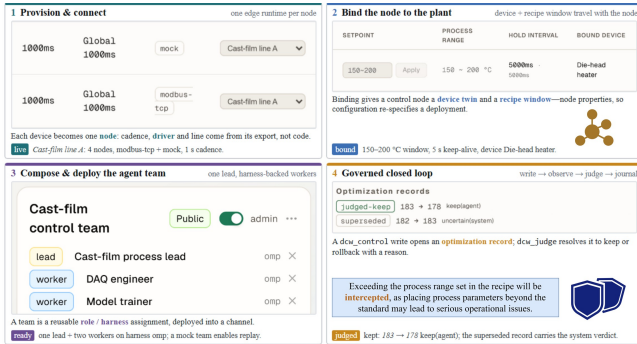


Fig. 4. The implemented path on the running operator surfaces: (1) provisioned nodes carry cadence, driver, and line assignment; (2) a binding attaches the node to its device under the active recipe window and hold interval; (3) a channel deploys one lead and two workers (Fig. 3's four roles are logical and map onto lead and worker instances); (4) a governed closed loop opens an optimization record and stores a verdict. Running UI/API surfaces, not benchmark outcomes.

Readback, where supported, closes the command path, not the process-response path.

A successful agent or manual write installs a post-write hold window (`writeLockSeconds`, default 30 s): the loop rests so the process can respond before the next proposal; a further agent-origin request inside the window receives HTTP 429 `WRITE_FREQUENT`. The details are narrow: the lock installs only after driver success, recipe-dispatch and rollback writes are exempt, zero disables it, and it is per node and agent-origin only—a pacing for loops whose response window may be comparable to the sampling cadence, not a rate limit on the

plant. `AW_BENCH_MODE=1` admits harness-declared ablation arms on the manual route that relax the soft window (the Stage-1 gate of Fig. 3), the readback check, or the hold—the hard engineering range is never bypassable.

Four governed routes—agent tools, human-origin writes, recipe dispatch, and rollback compensation—reach the same driver through one admission and journaling entry point, the hard range included; recipe dispatch adds run context, its values validated against the recipe window at save time. Hold heartbeats are the one non-governed re-dispatch—periodic rewrites of an already-admitted value, sharing runtime write exclusion without re-passing admission or conferring member authority. Line start installs active context before recipe parameters dispatch individually, and line stop clears context and closes retained open records. These multi-step operations are not atomic with device state.

C. Observation, Judgment, and Compensation

An accepted write opens an optimization record; the verdict tool `dcw_judge` records keep, rollback, or uncertain plus a reason; the verdict, stored with the record it closes and the response window actually queried, performs no actuation and does not independently validate the evidence, so a scripted keep can coexist with a missed objective. Verdicts are attributable—the actor and timestamp travel with them; corrections amend the stored reason visibly rather than replacing the verdict, and compensation opens a new record that references the one it reverses.

Compensation is a new, separately admitted write that may fail or partially complete; its dispatch is read-back verified, and a rollback record settles to rolled back only once the compensating value is confirmed at the device, while recipe rollback restores a definition, not physical PLC values. These separations keep command status, record status, and process recovery independently inspectable (Fig. 4).

The backstop is the platform's safety net for writes that no verdict has closed: when the recorded value drifts out of its window, the platform rolls the write back on its own. The stored policy waits at least 120s, checks at most every 30s, and triggers after three returned points or averages leave the window; recovery executes only while fewer than two retained rollback records stand on the node—past that point the backstop still judges and records but defers actuation to the approval policy that governed the write under review, so rollback loops cannot form. The cadence is neither a deadline nor a formal run-time assurance bound.

D. Human Approval Boundary

Manual bindings insert the timed approval wait of the life cycle above before an agent setpoint or compensation request; automatic bindings omit the wait but keep the checks. The pending request shows member, node, value, interval, and expiry; approval reserves neither interval nor device state, so binding and admission are re-checked at execution. The wait fails closed: an expired request resolves as refusal and does not actuate, with expiry and refusal recorded; duplicates are suppressed. Selecting the approval-free mode is itself a user-side, line-permission configuration action—agent identities hold no binding-administration capability—so an agent cannot reclassify its own writes as approval-free.

The decision endpoint authenticates a user but does not enforce a line-scoped approver role, and reads, verdict recording, and some configuration changes do not use physical-write approval—workflow mediation, not segregation of duties or certified human supervision; a recipe switch between approval and execution re-admits the value against the new window rather than re-presenting it to the approver (Section VI).

E. Design Rationale and Invariants

The strategy engine is replaceable rather than trusted as a control primitive: the stable plane keeps bindings, semantic I/O, evidence, approval, compensation, and backstop handling, while a scenario supplies blueprint, limits, objective, response window, and policy. Two rules enforce that boundary: scenario code declares and configures but never executes plant I/O, and the platform never infers a process objective—so every write still passes admission, dispatch, and evidence capture, and records can be re-evaluated against a revised predicate without rewriting history.

V. EXPERIMENTAL EVALUATION

The evaluation asks whether one governed supervisory path can be exercised, inspected, and repeated across heterogeneous industrial jobs, not whether one optimizer beats another.

Four questions organize the evidence: protocol closure and governance attribution (Q1), the AgentTeam workflow (the lead-worker team runtime of Fig. 3) on one node and on a multi-node line (Q2), the deterministic cast-film loop (Q3), and attainment for four process missions (Q4), each instantiating the path of Fig. 4. Q1–Q4 map onto the contributions just listed—Q1 exercises the integration boundary and attributes the governance mechanism, while Q2–Q4 occupy the strategy slot from scripted policies to concurrent missions.

A. Testbed, Evidence Snapshot, and Scope

The snapshot is integrated run 20260921184204-101c (seed 42, commit e692df1, harness 9f2756dce4a4cbb3, completed September 21, 2026, 19:08 UTC), archived with raw reports and mission logs; ablation companion 20260921181234-e11ite. Scripted external policies call the governed tools under a deployed member identity; two live agent-engine turns check that the same slot accepts an engine (archived trace headers record the harness, provider, and model: omp, glm-5.3-flash). The standing single-node preset is a cast-film extrusion line tuned through the die-head heater setpoint (objective J is the blueprint's weighted score over thickness conformance, defect rate, energy, and throughput under melt-temperature and pressure constraints—larger is better); each scenario contributes one trajectory from its seed-42 initial state.

B. Test Procedure and System-Level Profile

The procedure is a 20-phase integrated suite scoring 83/83 checks with no warnings or failures—an author-defined regression summary, not an external certification—probing one claim per phase, from preset loading against the offline optimum ($J^* = 89.894$) and per-protocol interception through the record, recipe, approval, and parameter-layer drills to the controller benchmark, second preset, backstop drill, and campaign (Section V-F).

Figure 5 reports where the time goes: Panel (a) shows each phase's wall time (log scale). Deterministic governance steps—admission, binding checks, record writes, recipe lifecycle—complete in milliseconds to seconds, so the platform's per-intervention cost is validation logic; the long tail sits in the replaceable strategy slot (the live engine turns take 156.3s and 637.3s) and the observation cadences (the backstop drill spans 134.7s, firing at 130.2s). Panel (b) is the transparency tier: the closed-loop objective after two writes, normalized by J^* ; all three seeds land within 3.5 percentage points of the oracle (Section V-E).

C. Q1: Protocol Closure and Governance Attribution

On the five protocol lines the suite stores 55 DAQ samples through real drivers, including a Modbus RTU-over-TCP satellite; governed writes are accepted and read back on Modbus TCP, OPC UA, MQTT, and HTTP (per-line medians 47.8ms, 19.8ms, 19.3ms, and 32.9ms). Four of the five lines are writable—the Modbus RTU-over-TCP satellite is acquisition-only in this deployment. Because one admission, record,

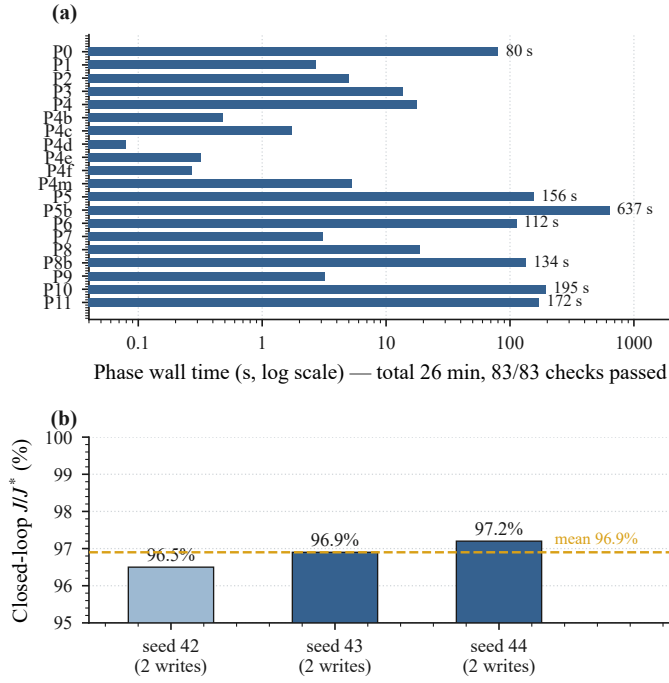


Fig. 5. Run 20260921184204-101cat system level. (a) Phase wall time (log scale); bar labels are harness phase identifiers. (b) Closed-loop objective J after two governed writes as a fraction of J^* , per seed, with no write rejected; the annotated mean is 96.9%. Panel (a) is environment telemetry, not a service-level bound.

and verdict path serves every request, the residual spread is consistent with transport differences. Each writable line rejects 6/6 configured out-of-window probes while accepting its legal write, so all 24 probes are intercepted; a dedicated window-interlock drill (the Stage-1 gate of Fig. 3) adds decision-boundary probes at the window edge and just outside it, and exercises both admission branches, line running and line stopped—the interlock discriminates rather than merely blocks.

Two drills then separate truth paths that one success flag would collapse: a frozen-measurement drill accepts a write and a consistent readback while the measurement stays out of window, and the backstop records a system-judged rollback 130.2 s later—readback closes the command path, the backstop closes the record, and neither is a settling time. A second preset with different signals and dynamics is then commissioned by configuration alone (zero framework code changes); across both presets, 33 configured probes are rejected on seven writable lines—six per line on the first, three per line on the second—the same admission code discriminating on signal maps it had not previously exercised.

The parameter layer passes 4/4 checks and makes the interval geometry of Section IV-A observable: a four-layer chain (node [50, 200] rpm, parameter [143.7, 156.3] rpm, product [146.22, 153.78] rpm, recipe [141, 159] rpm) narrowing to one admitted interval, each rejection attributable to its excluding layer. A manual binding instead suspends a request visibly and unblocks it on approval, after which the write takes effect at the device (automated round-trip latency 461 ms): workflow mediation, not physics.

Table I attributes the rejection rather than assuming it,

TABLE I
GOVERNANCE ABLATION ON COMPANION RUN
20260921181234-ELITE (THREE DETERMINISTIC REPETITIONS WITH IDENTICAL COUNTS; 18 PROBES PER ARM: 12 HARD-RANGE AND 6 WINDOW-CLASS).

Arm	Rejected	Executed breaches	False blocks
Full governance	18/18	0	0
No window interlock	12/18	6	0
No readback check	18/18	0	0
Both relaxed	12/18	6	0

on a standalone admission-pipeline fixture kept outside the scenario simulators; counts are identical across the three deterministic repetitions, and the per-request latencies archived for every arm overlap at this resolution—we claim only that no overhead is visible on this fixture. Full governance and the no-readback arm reject 18/18 probes, whereas removing the window interlock, alone or with the readback check, lets the six window-class probes execute and be journaled. Two readings follow. First, prevention is the window interlock’s job: the readback check contributes nothing to interception—it closes the command path after execution—yet is retained because it makes a written-but-unconfirmed command visible. Second, attribution survives removal: the six executed breaches still appear in the journal, so a relaxed configuration degrades from prevention to detection. The hard range is structurally unbypassable and the soft window is its removable part; no arm records a false block.

D. Q2: The AgentTeam Optimization Workflow and Its Results

The workflow ran end to end on the cast-film line and then on a larger BOPET line (Section III-D); a scripted policy rather than a language model issued the writes in both missions (the two live engine turns follow at the end of this subsection). On the single-node mission one write sufficed: the die-head heater setpoint moved from 200 to 204.704 °C and the observation settled at 204.70 °C against 204.704 ± 0.75 °C, the journal attributing the change to the agent identity, the task closing COMPLETED.

The BOPET mission is the multi-node case: nine devices and 49 signals (30 writable, 19 readable), all created by the *ensure* pass, whose driver tests passed on all nine endpoints. Thickness started at 28.10 μm against 25.0 ± 0.7 μm . Three governed writes on three distinct nodes—casting speed 32 $\rightarrow$ 33.8 m/min, fast-roll speed 118 $\rightarrow$ 120.3 m/min, and rail width 3000 $\rightarrow$ 3035 mm—brought it to 25.66 μm . Each write opened a record, received *keep*, and left an agent-attributed journal entry, while melt temperature stayed near 291 °C and the parent task reached COMPLETED. In both missions the governed-write count matched the number of accepted setpoint changes—by construction, since each write traverses the same admission, record, and verdict steps regardless of line size.

In the bounded turn the agent read the node’s semantic card, sampled a baseline through *daq_query*, wrote 197.48 °C, waited, resampled, and closed the record with *keep*, finishing in 156.3 s and storing one lesson: the acquisition-limit flag can lag the live value. The goal-driven turn took 637.3 s: it queried the journal and its own memory, found neither,

measured $53.79 \mu\text{m}$ at 150 rpm against a self-selected target of $52.0 \pm 0.8 \mu\text{m}$ (the governed path, not a predicate, constrains this turn), inferred from mass conservation ($h \propto N/v$) that screw speed should fall by about 5 rpm, applied the change in steps, judged each write on measured evidence, refined its gain estimate from 0.22 to $0.36 \mu\text{m}$ per rpm, and settled thickness at $51.7 \mu\text{m}$; it also saved the recipe—authorship attributed and audit-logged, activation role-gated.

E. Q3: The Deterministic Closed Loop

Seeds 42–44 each issue two governed writes with no rejection, raising the cast-film objective J (larger is better) from $J_0 \approx 67.2$ to $J/J^* = 0.965, 0.969$, and 0.972 on the three seeds, against the simulator’s offline reference $J^* = 89.894$ obtained by exhaustive grid search over its steady-state map. The residual gap to unity is expected: J^* is an oracle with full model knowledge, which an online policy seeing only process observations cannot in general match. What the benchmark establishes is narrower: the governed path passes a conventional controller’s writes through unmodified—admissible, journaled, and read back—and no ungoverned performance arm was run, so the path’s transparency is shown for admission and evidence handling rather than measured as a performance delta.

F. Q4: Four Process Missions

The campaign runs four lines concurrently, each provisioned from a blueprint by the *ensure* pass and given one channel; all four attain their archived predicates, evaluated from subsequent observations, not the task state. The controlled quantity moved from 31.25 to 32.17 g, 0.21 to 3.50 mg/L, 134.52 to 100.08 HV, and 28.10 to $25.66 \mu\text{m}$; mission clocks (26.48 s, 169.27 s, 102.76 s; the BOPET clock was not retained) are simulator wall time under accelerated dynamics, not physical durations. Figure 6 archives the trajectories: the curve is the observed controlled variable, the hatched band its window, and the filled terminal marker the predicate-satisfying observation; each mission iterates the same loop—read, write inside the admitted interval, observe, judge—so one governed loop produces all four trajectories.

Panel (a): injection molding is the minimal case—two writes raise the hold pressure from 45 to 58.2 bar and the hold time from 6 to 10.64 s, and the policy stops at 32.17 g inside 32.5 ± 0.35 g in 26.48 s, with the flash and sink guards inside their limits.

Panel (b): wastewater treatment, the longest mission, has two stages. Aeration, caustic, and coagulant dosing bring dissolved oxygen from 0.21 into 3.4 ± 0.6 mg/L under the effluent guards on COD, ammonia, total phosphorus, pH, and turbidity; the policy then lowers blower frequency and caustic dose, ending after 15 writes at 3.50 mg/L in 169.27 s with the simulator-internal compliance-cost index (Fig. 6b), the scenario’s operating-cost proxy, down from 181.5 to 179.1. Two blower steps are the most informative events: both writes were admissible and journaled, the post-write observation lost compliance margin, and the verdict stage reverted them—rejection of a bad proposal lives in the evidence, not in the admission gate.

Panel (c): annealing starts far outside the hardness window (134.52 HV at 140 m/min). Raising zone-3 temperature to 850°C brings hardness to 99.46 HV; one zone-2 trim and two line-speed steps then hold 95 ± 6 HV while raising line speed by 17.1%, and a further attempt at 176 m/min is reverted for leaving the margin. All 5 governed writes are journaled; the panel’s baseline-indexed axis plots the states they produced—the fifth (the reverted 176 m/min attempt) returned the process to the prior state, so it shares that position and is distinguished only in the journal. The guard is multi-objective—hardness and throughput must hold together—and the predicate holds in 102.76 s only because the margin-violating speed was undone, on record.

Panel (d): BOPET is the nine-device mission of Section V-D executed under the campaign harness, moving thickness from 28.10 to $25.66 \mu\text{m}$ inside $25.0 \pm 0.7 \mu\text{m}$. In this mission completion and the predicate agree, yet they remain separate records, so a later missed objective would still be visible.

G. Cross-Scenario Reading

Read together, the four missions bound what the framework contributes: platform-side behavior is invariant while the write economy—the governed-write count each scripted policy needs—varies by more than a factor of seven (2, 15, 5, and 3 writes) and the mission clock by more than six (26.48 s to 169.27 s over the three retained clocks), both produced by construction in the scenario-specific slot. The three reversions (two blower steps, one speed step) show caller-side verdicts doing real work; that all four terminal markers lie inside their bands is an observed outcome, not a structural property. Margins are narrow but on record: 32.17 g sits 0.02 g above 32.15 g, 100.08 HV 0.92 HV below 101 HV, and $25.66 \mu\text{m}$ 0.04 μm below $25.7 \mu\text{m}$; the compliance-cost index ends 1.3% lower.

VI. DISCUSSION AND LIMITATIONS

A. Industrial-Informatics Interpretation

The principal result is that one supervisory path carried four different jobs while keeping command closure, the submitted verdict, task state, and process attainment as separate records; relative to the nearest neighbors of Section II, the differentiator remains production-scoped authority and evidence on a physical write path. In run 20260921184204-101c all four scenario predicates hold, and the governed loop reaches J/J^* between 0.965 and 0.972.

B. Validity and Reproducibility

The snapshot fixes run identifier, seed, commit, harness fingerprint, and raw artifacts, so the reported counts can be re-derived rather than trusted. Three qualifications bound what they support. First, the evidence is simulation-tier: the simulators are deterministic per seed but are not validated digital twins, and no physical-device campaign, operator study, Tennessee Eastman Process (TEP)/Secure Water Treatment (SWaT) replay, or external-framework comparison is reported. Second, the controller optimum and the scenario predicates

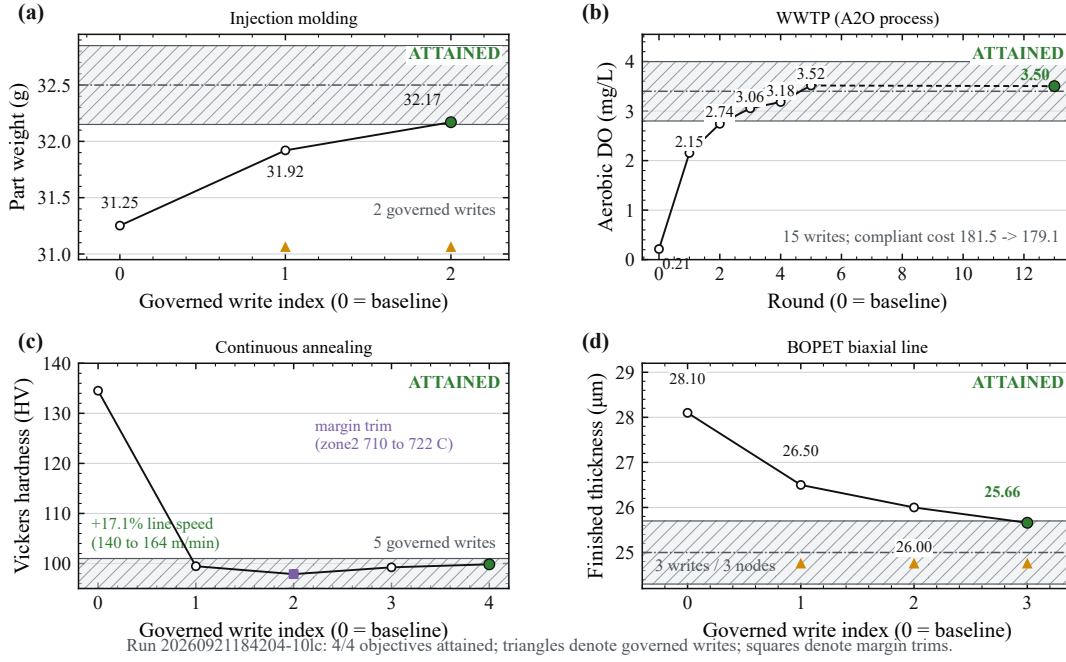


Fig. 6. Archived trajectories of the four missions in run 20260921184204-101c. Hatched bands are objective windows, the dash-dotted line the target, filled terminal markers the predicate-satisfying observations. Panel (a) marks governed-write rounds with triangles, Panels (c) and (d) index points by governed write, and Panel (b) tracks dissolved oxygen through the mission’s stages (its round axis includes the 15 governed writes) with write count and cost index annotated; the square in (c) marks the zone-2 margin trim. All four terminal markers lie inside their objective window. Curves are simulator observations under scripted policies, not physical measurements.

are internal oracles, the ablation has three deterministic repetitions, and initial state, asynchronous sampling, response windows, and simulator timing still influence deterministic policies, so results support mechanism attribution rather than statistical inference; latencies are single-host observations, not service-level bounds. Third, several capabilities of Section III are implemented but not exercised—memory retrieval quality, plugin hot-reload, alarm-chain timing, engine interchange—and the live turns share one provider and model, their memory call returning no stored entry, so that layer is exercised rather than validated; the four process models broaden structural variety, not transfer.

C. Safety Positioning and Human Oversight

The objection that a language-model team should not “take over” production applies to a design AGENTWORKSHOP does not propose: fast regulation, hardware interlocks, and safety functions remain in the PLC; Section IV-D bounds what an agent can change; and prompts, retrieved documents, and memory are untrusted text, not authority [42, 43]—the proposer is confined to per-binding scope and journaled, attributable actions: a bounded blast radius, not a protection guarantee. Binding scope, admission limits, readback, and the backstop run with or without human approval, bounding approval fatigue and automation bias [35, 36, 44, 45]; approval is a revocable per-binding policy; because admission is re-checked at execution, a stale approval cannot carry an out-of-window value nor outlive a context change.

D. Deployment Boundaries

Execution and recovery are non-atomic: bookkeeping may fail after actuation, the post-write hold guards write frequency

not stability, readback agreement need not imply process recovery, and multi-node compensation may partially succeed; oscillation, storage failure, clock error, worst-case recovery time, cross-run time-range queries (records lack run identifiers), and concurrency beyond the four-line campaign remain unevaluated.

In production, bindings govern sanctioned tools, not arbitrary privileged-engine I/O, so least privilege, endpoint security, and IT/OT separation remain necessary [29, 46], and independent hardware interlocks and any required IEC 61511 protection must stay in place: AGENTWORKSHOP is neither a safety instrumented function nor a substitute for plant protection [47]. The approval wait already fails closed on expiry, but line-scoped approver authorization is still missing, so production deployment needs segregation of duties and alarm-path failure tests [34].

VII. CONCLUSION

AGENTWORKSHOP contributes a governed supervisory execution architecture rather than a new process optimizer: one stable path from agent-team identities through semantic nodes, production context, governed writes, readback, observations, verdicts, and compensation, exercised by a 20-phase suite, agent-team missions, a three-seed loop, and four missions—all 83 checks passing and all four mission predicates holding in the reference run. Physical-plant deployment, operator studies, and external-framework comparison remain open; until then, claims of optimization superiority, autonomous control, and safety assurance stay unsupported.

DATA AVAILABILITY AND DISCLOSURE

Artifacts are archived in `bench/results/`: main run `bench/results/20260921184204-101c`, ablation companion

bench/results/20260921181234-e11ite. Reproduction needs the simulator source and reset state; the revision artifact will pin the currently missing simulator-tree fingerprint. AI assistants supported editing and checking under author direction; the authors remain responsible for all claims and data.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Sci. Rep.*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, "Assessment of a large language model based digital intelligent assistant in assembly manufacturing," *Comput. Ind.*, vol. 162, 2024, art. no. 104129.
- [3] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Trans. Softw. Eng.*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakih, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.
- [7] P. Leitão, "Agent-based distributed manufacturing control: A state-of-the-art survey," *Eng. Appl. Artif. Intell.*, vol. 22, no. 7, pp. 979–991, 2009.
- [8] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, "CoMA-IGK: LLM-driven multiagent framework for automated construction of industrial knowledge graph," *IEEE Trans. Ind. Inform.*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [9] P. Liu, L. Qian, X. Zhao, and B. Tao, "Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly," *IEEE Trans. Ind. Inform.*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [10] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *Proc. IEEE ETFA*, 2024, pp. 1–4, arXiv preprint arXiv:2405.18092.
- [11] M. S. Gill, J. Vyas, A. Markaj, F. Gehlhoff, and M. Mercangöz, "Leveraging LLM agents and digital twins for fault handling in process plants," in *Proc. IEEE ETFA*, 2025, pp. 1–8, arXiv preprint arXiv:2505.02076.
- [12] R. Wu, J. Ai, and T. Li, "InstruMPC: A human-LLM-in-the-loop framework for context-aware control," in *Proc. 2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025, pp. 172–179.
- [13] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [14] LangChain, Inc., "LangGraph," [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024, accessed: Sep. 20, 2026.
- [15] CrewAI, Inc., "CrewAI," [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024, accessed: Sep. 20, 2026.
- [16] Anthropic, "Model context protocol," [Online]. Available: <https://modelcontextprotocol.io>, 2024, accessed: Sep. 20, 2026.
- [17] Linux Foundation, "Agent2Agent (A2A) protocol," [Online]. Available: <https://a2a-protocol.org>, 2025, accessed: Sep. 20, 2026.
- [18] International Society of Automation, "ISA-88.00.01-2010—batch control—part 1: Models and terminology," Standard, 2010.
- [19] —, "ANSI/ISA-95.00.01: Enterprise-control system integration," Standard, 2010.
- [20] International Organization for Standardization, "ISO 23247: Digital twin framework for manufacturing," Standard, 2021.
- [21] International Electrotechnical Commission, "IEC 63278-1:2023: Asset administration shell for industrial applications—part 1: Asset administration shell structure," International Standard, 2023.
- [22] J. Moyne, E. del Castillo, and A. M. Hurwitz, *Run-to-Run Control in Semiconductor Manufacturing*. Boca Raton, FL: CRC Press, 2000.
- [23] A. Gautam, M. R. Aryal, S. Deshpande, S. Padalkar, M. Nikolaenko, M. Tang, and S. Anand, "IIoT-enabled digital twin for legacy and smart factory machines with LLM integration," *J. Manuf. Syst.*, vol. 80, pp. 511–523, 2025.
- [24] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber-physical systems," *IEEE Trans. Ind. Inform.*, vol. 21, no. 3, pp. 2719–2728, 2025.
- [25] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Ji, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Adv. Eng. Inform.*, vol. 69, 2026, art. no. 103888.
- [26] Z. Xiang *et al.*, "GuardAgent: Safeguard LLM agents by a guard agent via knowledge-enabled reasoning," 2024.
- [27] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," 2025.
- [28] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025.
- [29] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2013.
- [30] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [31] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Trans. Autom. Control*, vol. 62, no. 8, pp. 3861–3876, 2017.
- [32] L. Sha, "Using simplicity to control complexity," *IEEE Softw.*, vol. 18, no. 4, pp. 20–28, 2001.
- [33] International Society of Automation, "Ansi/isa-18.2: Management of alarm systems for the process industries (iec 62682)," Standard, 2016.
- [34] Engineering Equipment and Materials Users Association, "EEMUA 191: Alarm systems—a guide to design, management and procurement," Guide, Edition 3, 2013.
- [35] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern., Part A*, vol. 30, no. 3, pp. 286–297, 2000.
- [36] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [37] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, arXiv preprint arXiv:2308.03688.
- [38] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [39] F. F. Xu, Y. Song, B. Li, Y. Tang, K. Jain, M. Bao, Z. Z. Wang, X. Zhou, Z. Guo, M. Cao, M. Yang, H. Y. Lu, A. Martin, Z. Su, L. Maben, R. Mehta, W. Chi, L. Jang, Y. Xie, S. Zhou, and G. Neubig, "Theagentcompany: Benchmarking LLM agents on consequential real world tasks," *arXiv preprint arXiv:2412.14161*, 2024.
- [40] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Comput. Chem. Eng.*, vol. 17, no. 3, pp. 245–255, 1993.
- [41] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [42] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security*, 2023, pp. 79–90.
- [43] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>
- [44] J. D. Lee and K. A. See, "Trust in automation: Designing for appropriate reliance," *Hum. Factors*, vol. 46, no. 1, pp. 50–80, 2004.
- [45] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Hum. Factors*, vol. 52, no. 3, pp. 381–410, 2010.
- [46] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [47] International Electrotechnical Commission, "IEC 61511-1:2016: Functional safety—safety instrumented systems for the process industry sector—part 1: Framework, definitions, system, hardware and application programming requirements," International Standard, 2016.